

Assignment 2 Coding Documentation

1. Pacman Environment

For Assignment 2 we will work with an existing game environment that implements “Pacman Capture The Flag”. This supporting application provides us with the necessary hooks to implement, load and then test our customised agent controllers.

1.1 Simulator

capture.py is the command line entry point for the simulator. This program requires some arguments to specify the team and the rules of the game. You can find the full list of arguments accepted by the simulator with the following command (-h means help):

```
python capture.py -h
```

We highlight a few particularly important arguments, of which you should be aware. Bracketed [terms] indicate parameters:

- -r [RED.py: Default baseline.py]
Path to red team python implementation
- -b [BLUE.py: Default baseline.py]
Path to blue team python implementation
- -l [LAYOUT: Default ./layouts/defaultCapture]
The description map layout to use for the game. The LAYOUT parameter can be a filename, in which case the layout is loaded from disk. There are several pre-generated map files in the **layouts** folder. Alternatively, the value RANDOM can be given, which will generate a new random maze. We can also start from a particular random seed using the format RANDOM<seed>; e.g., RANDOM23.
- -q
Display minimal output and no graphics
- -Q
Same as -q but agent output is also suppressed
- -i [MAX_MOVES: Default 1200]
Specify a limit to the game by giving a maximum for the total number of moves. The game will end after this many moves have been executed, across all agents.
- -n [NUMGAMES: Default 1]
Number of games to play.

1.2 Run the game

The following command runs the a game between **staffTeam.py** implementation and **berkeleyTeam.py** implementation:

```
python capture.py -r staffTeam.py -b berkeleyTeam.py
```

The simulator will start and load implementations from red and blue teams and each team has two agents. It also calls some preparation functions for each agent (we will detail these below). Remember to try different map layouts during training and testing.

Once the game starts, agents take actions one by one, turn by turn (the order is agent id; agents with id 0, 2 are one team and agents with id 1,4 are another team). The simulator will call the **chooseAction** function from the given implementation at each turn of an agent. The function must return one of the cardinal directions, "North", "South", "East", "West", or it can return "Stop" as the action (the agent waits at current location). The simulator immediately executes the action and then it moves to the next agent and calls its **chooseAction** function. Time advances by one timestep after every agent has moved.

2. Implement Your Agent

You can find three examples in the Pacman Capture the Flag project.

The most important is **myTeam.py**, which contains the decision-making code for your agent and which you will modify. We include a simple baseline implementation as a concrete reference. The baseline relies on **myTeam.pddl**, which is a simple example of how we can PDDL to guide the high-level actions of the agent (defend home, attack for food, escape from enemies etc). The low level actions of the agent (what exact action to do to complete a high level action) is guided by a (partially implemented) Q-learning model, in which the decision is based on a set of weights and features.

Two other agent controllers of which you should be aware:

- **staffTeam.py**
This is a copy of **myTeam.py**, so that you can compare your improvements with the existing baseline.
- **berkeleyTeam.py** is a default controller included as part of the Pacman Capture the Flag game environment (and implemented by the developers, at UC Berkely).

You are free to create your implementation from scratch without relying on the staff baseline implementation.

2.1 myTeam.py

This is where you should start your implementation. Most of the implementation for the baseline controller can be found in the **MixedAgent** class (which we discuss below). In addition, **myTeam.py** contains some important initialisation code, **createTeam**, and various constants, of which you should be aware.

createTeam

The environment uses **createTeam** function to create two agents from your implementation. You can specify the name of the class of your implementation in the first and second argument to specify exactly which class for which agent.

Usually you don't need to modify the **createTeam** function. But if you want two agents with two different implementations, you can modify the function. Or if you want to pass some options to your agent implementation from CLI, read the function documentation of how to do that.

The default team instantiates both agents from the class 'MixedAgent', which is a baseline implementation you can modify (see below for more details). Alternatively you can create your own agent implementation from scratch. All agent implementations need to inherit from the class **CaptureAgent** provided by the game environment (see **captureAgents.py**).

Constants:

- **BASE_FOLDER** stores the absolute path for the folder of myTeam.py. If you need to specify paths anywhere in the code (e.g., read a text file) always give it paths relative to **BASE_FOLDER**
- **CLOSE_DISTANCE**, **MEDIUM_DISTANCE**, and **LONG_DISTANCE** are used for high-level planning with PDDL (they help the planner reason about distances between things). These constants are discussed in section 2.4 and 2.5. You can modify them to a number you think is reasonable.

The MixedAgent Class

This provided staff baseline agent class is called **MixedAgent**. This class uses PDDL for high-level planning and Q learning for low-level planning (features for both attacking and defending). You should read and understand this code if you intend to extend it. Alternatively you can also begin from scratch, by overwriting myTeam.py with the content of emptyTeam.py.

2.2 Preparation

When the game start, the simulator will initiate each agent and do some preparation work:

- **registerInitialState** is called only at the start of each game. If you need to prepare some data before the start of the game, do that here.
- **self.pddl_solver** is where the PDDL solver initiates, **make sure the path for PDDL domain file correctly points to the one you want to use!**
- **final** function is called at the end of the game. If you need to do something at the end of a game, do them here.
- Note that class variables, like **QLWeights** (in the baseline agent), are shared and accessible by all agents created from the same class.

2.3 Decision Making Process

Each agent must implement a **chooseAction** function. This function is responsible for the decision-making process and on completion it must return a concrete move action to the simulator.

This is the most important function in your entire implementation. You should read the code line by line, understand what each function does, and what other functions called by these functions do.

As described in the workflow diagram in Assignment 2 Specification, **chooseAction**:

1. computes a high-level plan if does not exist or next high-level action not applicable,
2. select the next high-level action from the high-level plan,
3. computes a low-level plan that targeting the high-level action if does not exist or action cannot be executed,
4. select the next low-level action and return to environment for execution.

2.4 Implementing High-Level Planning

In **MixedAgent**, the high-level planner generates high-level PDDL problems programmatically (instead of always loading a PDDL problem from a text file), and solves the problem based on a simple domain model. We briefly describe the main parts of this baseline implementation.

- **myTeam.pddl**
This file contains a list of potentially useful PDDL predicates that can be used for high-level planning. Also concrete specifications of a few simple high-level actions.
- **get_pddl_state**
This function is where we convert game state data into PDDL data. The function collects `:init` state expressions and object expressions for PDDL problems.
A **state expression** here is a **TUPLE**, for example tuple `("food_avaliable",)` means in a PDDL problem you write `("food_avaliable")` in `:init`. **Pay attention to the "comma"** in the tuple, without the comma python thinks it is just brackets. A tuple `("is_pacman", "a1")` means you write `("is_pacman a1")` in `:init`.
An **object expression** here is also a **TUPLE**. A tuple `("a1", "current_agent")` is the same as you write `"a1 - current_agent"` in PDDL `:objects`.
When collecting states, constants introduced in section 2.1 will be used to define what is close, medium or long. e.g. If **LONG_DISTANCE** = 25, the expression `("enemy_long_distance", "e1")` indicating the noisy distance to "e1" returns a number larger or equal than 25.
- **getGoals**
This function selects the applicable goal function with highest priority and returns the corresponding state expressions for `:goal` of PDDL problem.
Here a state is still a tuple. But all states with "not" in PDDL problem goes to **negativeGoal**, e.g., `("not (food_avaliable)")` in the `:goal` of PDDL problem is a tuple

("food_available",) in **negativeGoal**. All expressions without "not " go to **positiveGoal**.

- **stateSatisfyCurrentPlan**
This function checks if there exists a plan, can we continue to execute current high-level action in the plan or can we move to the next high-level action in the plan.
- **getHighLevelPlan**
This function solves the PDDL problem and return a high-level plan.
A high-level plan is a list of Action, pddl_state tuple. See definition of Action in lib_piglet.utils.pddl_parser and definition of pddl_state in lib_piglet.domains.pddl

Tips for improving the high-level baseline

The existing domain model (**myTeam.pddl**) provides only a few simple actions that rely on a small set of "basic" predicates. Other "advanced" predicates are also available. You can use these to extend the existing actions or to create new high-level actions that allow for more sophisticated high-level plans.

The model currently distinguishes between two predicate types:

- **team** type predicates, which are used to reason about the current agent and its ally. Also to track the progress being made by the team in the game (e.g., by tracking the score of the game)
- **enemy** type predicates, which are used to reason about the enemy team.

Although the baseline makes available a variety of convenient game information you may notice the available data is still only a small subset of the information available in the game state. You may find it useful therefore to introduce your own new predicates and to track and reason about other game-related information which is not tracked by the model. In this case you will also need to modify **get_pddl_state** to collect this additional information.

2.5 Low-level Planning

For low-level planning, students can choose either Q learning or Heuristic search to plan low level actions. The important functions to be aware of here are the following:

- **posSatisfyLowLevelPlan**
This function checks if there exists a low-level plan and does agent's current location still sticks to the plan.
- **getLowLevelPlanQL**
This function computes a single action low level plan (a list with only one element) using reinforcement learning. An element in a low-level plan is a tuple of action, and target location coordinates.
- **getLowLevelPlanHS**
This function computes a low level plan (a list of tuples of action and target location) using heuristic search. An element in a low-level plan is a tuple of action, and target location coordinates. You can call this function in the low level plan section of

chooseAction instead of **getLowLevelPlanQL** to compute a low-level plan using heuristic search.

Each high-level action should have its own low level planning strategy to successfully achieve its target.

2.6 Implementing Heuristic Search Low Level Planning

The **MixedAgent** baseline uses Q-learning. But you may decide the best approach for your low-level planning is heuristic search. In this case refer to the function **getLowLevelPlanHS**, which is not implemented yet. You should finish the implementation based on your knowledge from week 1 to week 6.

To implement a heuristic-search-based planner you will need to think about:

- Given high level action, how to select/compute a location or target for the agent.
- How to compute a plan to reach that location/target
- How to return the plan in the form of a list of tuples of action and location, which the simulator can execute.

Remember that actions are "North", "South", "East", "West", or "Stop". Location is a coordinate of the targeting location of the action..e.g. [("North",(1,1)), ("East",(2,1))]

For information about maps, obstacles, and food locations, refer to section 3.

2.7 Implementing Q-Learning Low Level planning

The default low-level planner **getLowLevelPlanQL** uses approximate Q learning (refer to the lecture material) to compute next movements. It classifies existing high-level actions to three categories with three low-level planning strategies (**Note, you should improve the low level planning by having each high level action mapped to their own low level planning strategy**). You could implement your own learning model for low level planning, not limited to approximate Q learning.

The current low-level strategies have many drawbacks, you should run the game and observe these drawbacks by watching what happens in the visualiser.

For each strategy, the **getLowLevelPlanQL** prepare the

- get reward function
- get feature function
- weights

for approximate Q learning update and evaluation.

The current offensive strategy has reward function, feature function, weights all prepared (but very naive, with many improvement spaces). The reward function for defensive and escape strategies are not implemented and their corresponding learn rates are set to 0 to prevent any weight update.

The weights of a strategy is stored in the class variable **QLWeights**. **QLWeights** comes with default values in class variable definition, and will be loaded(if file exist)/stored to disk at the beginning/ending of each game run. See `registerInitialState` and `final` function for related codes.

You should improve the existing strategy by:

- Improve the feature function to give more/better (but useful and helpful) information of the agent for q learning.
- New features should have corresponding default weights in class variable **QLWeights** and delete the `QLWeightsFile` on your disk, in case old weights override the new one.
- Better reward function

When implementing a new strategy, you should:

- Implement a get feature function that collect features from the environment(from `gameState`, `CaptureAgent Convenience Methods`, and `AgentState`)
- Add default weights for the new strategy in class variable **QLWeights**, delete the `QLWeightsFile` on your disk, in case old weights override the new one.
- Design and implement the reward function.

Designing Feature Function

The approximate Q learning uses extract features from successor state, then sum and multiply each feature value with its corresponding weight to be Q Value, which reflects how good the successor state is. Then choose the action that leads to the best successor state(largest Q Value). Thus designing good feature functions to collect information from the environment is important.

Try normalising the value of different features into the same range on maps with different sizes.

Avoid features whose evaluation changes rapidly for small changes on the gameboard. For example, suppose we have a feature called `chance_of_losing_food` whose domain is in range `[0,100]`. The evaluation of this feature is zero until the agent is carrying food and then can suddenly change close to 100 if there's a nearby ghost.

A good general principle is that changes in feature values should be smooth between states. Otherwise the training will be difficult.

Refer to **`getOffensiveFeatures`** to see how staff implement feature function

Designing Reward Function

The reward function usually returns different negative values for the current state of the agent. You may only return positive values after many non-rewarding steps when something good/encouraged happens to the agent, e.g. agent returns food to home.

Refer to `getOffensiveReward` to see how staff implement reward function

Keep in mind that:

- There needs to be a correlation between the state information and the reward: the simpler the relationship, the easier/faster the model will find it.
- Sparse and binary rewards make the training problem long and arduous. Giving more information through the reward can tremendously increase the speed/accuracy of the learned Q-estimator.
- The longer the chain of actions, the more complex the Q-value will be to estimate.
- Avoid giving large penalties based on binary outcomes or contradictory outcomes. For example, you might decide to give a large penalty every time the agent is eaten by a ghost. But being eaten when carrying lots of food is better than being eaten when carrying little or no food. Applying a large penalty does not distinguish between these situations.
Another bad example is, if you design a feature that gives large rewards when the agent is carrying lots of food and similarly large penalties when the agent is eaten by a ghost, the overall information learned by the agent is zero.

Training the model:

There's a **self.training** attribute in the `registerInitialState`, if this is set to **True** the **updateWeights** will update weights before calculating Q value with **getQValue** function. If this is set to **False**, weights will not be updated, and random exploration will not happen.

There are some parameters regarding the training you should pay attention to:

- `self.epsilon = 0.1` Default exploration prob, which is also the chance to take a random low level action
- `self.alpha = 0.1` Default learning rate
- `self.discountRate = 0.9` Default discount rate.

If the training makes weights go in the wrong direction, you can delete the text file (specified in `QLWeightsFile`) stores QL weights, and restart the training. In this case, the `QLWeightsFile` does not exist and the program uses default weights stored in **QLWeights** as a starting point.

You should expect numbers in weights to change slightly in each update and become more and more stable during the training process.

Small learning rate helps to stabilise the weights but slows down the training speed.

You repeat the cycle of:

- adjust features
- adjust reward
- training
- see if weights get stable and the agent behaves as expected.

HINTS:

- During training record the “correction” value after each weight update. Check if the value has sudden huge changes and why these huge changes happen. Try eliminating these abnormalities by adjusting feature and reward implementation.
- You may want to train your low-level planner independent from high level decisions to focus on the training of a specific low-level planner. For example, when training a low-level planner for “attack” high-level action, you could disable the high level planner and always use “attack” as high-level action. The opposite team in the training game can focus on defence only.

Turn off the `shelf.training` when you submit the code to the contest server.

You can run pacman in silence mode with “-Q” argument and you can specify number of games with “-n NUMGAME”. This will allow you to simulate many games; e.g., with “-n 100” argument, it runs 100 games. You can replace 100 with another number you want. You also need to use “-l ./layouts/bloxCapture.lay” (replace the bloxCapture.lay to other maps) to train your agent on other maps in the “layouts” folder, or train on a random map by reading how to generate a random map with “python capture.py --help”.

3. Working with the Game Environment

In this section we outline some important details for how to obtain observations and other useful information from the game environment. Reading and comprehending the implementation of the game environment can be immensely beneficial to your implementation.

3.1 GameState

Any **gameState** variable you see in the implementation is an object of **GameState** class in **Capture.py**. It provides all the information of the environment for your current agent. It also provide a bunch of convenience methods to return information from the current game environment. **Read the methods of this class** to know what it provides. Using the convenience methods described in next section make it easier to retrieve information from **gameState**.

Refer to **get_pddl_state** method and those get features functions on how we use these methods.

3.2 Convenience Methods

There are a bunch of convenience methods in the implementation of **CaptureAgent** in **captureAgents.py**. You can call these methods in your implementation at any time to acquire information conveniently.

Read the codes in “Convenience Methods” section to know what kind of convenience methods the template agent class provide.

For example, if you find there's a function called **getFoodYouAreDefending** in **CaptureAgent** class, then you can call this function in your implementation by `this.getFoodYouAreDefending(gameState)` to get the foods that have an enemy next to them. So that, you know the location of the enemy although they are beyond your observation range.

Refer to **get_pddl_state** method and those get features functions on how we use these methods.

3.3 Grid/Map

Functions like **getFood** of **CaptureAgent**, **getWalls**, **getBlueFood**, and **getRedFood** of **GameState** returns a **Grid** indicating on each location if there's a food or obstacle.

A **Grid** is a 2-dimensional array of objects backed by a list of lists. Data is accessed

via `grid[x][y]` where (x,y) are positioned on a Pacman map with x horizontal, y vertical and the origin (0,0) in the bottom left corner.

For a **Grid** returned by **getWalls**, `grid[x][y] == True` indicates location x,y has a fixed obstacle.

For a **Grid** returned by get food related function, `grid[x][y] == True` indicates location x,y has a food.

asList() method of a grid will return a list of location coordinates that are True in the grid.

3.4 AgentState

Functions like `getAgentState` of **GameState** returns an object of **AgentState** class defined in `game.py`. This class contains the state of an agent, which includes Pacman, sacred timer, and food carrying. **Read the definition of this class.**

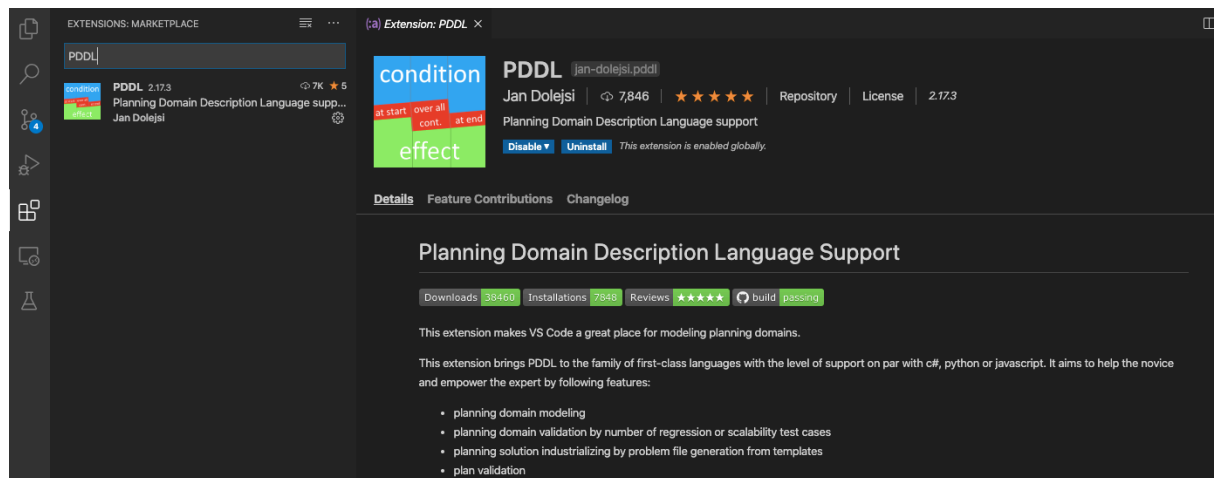
APPENDIX PDDL

Learn PDDL

- PDDL wiki: <https://planning.wiki/>
This website contains a detailed guide to PDDL and references to PDDL related terminologies.

Text Editor: Visual Studio Code with PDDL extension

1. Install PDDL extension by searching “PDDL” in the extension marketplace of Visual Studio Code.



2. The extension gives grammar highlights if you open a PDDL file.

Piglet PDDL Solver In Pacman

In pacman, we use an interface implemented in `lib_piglet.utils.pddl_solver` to solve problems generated programmatically in the agent implementation. See the corresponding implementation in **getHighLevelPlan** of `myTeam.py`

You can read its implementation and also the implementation of `lib_piglet.utils.pddl_parser` to know how it works.

Piglet PDDL Solver Supported Requirements

The piglet PDDL solver support:

- `:typing`
- `:strips`
- `:negative-preconditions`

Typing

```
(:types
  animal - object
  cat mouse - animal
)
```

In PDDL, every object belongs to a certain type. You can declare any type with `:types`. You can write PDDL without any types, but types make your model more clear. In this example, type `animal` is a subtype of the `object`. Type `cat` and type `mouse` have supertype `animal` and also inherit type `object` from type `animal`.

Constants and Variables

Variables can refer to any applicable object of some types. Variables are always written with a “?” as a prefix. For example: `(?c - cat ?m - mouse)` means a variable `?c` with type `cat` and a variable `?m` with type `mouse`.

In contrast, you can declare constant with `(:constants Tom - cat)`. `Tom` is a constant object of type `cat`.

Predicates

A predicate is an atomic statement that is used to express certain conditions in the logic of a planning problem. For example:

```
(door_open)
(at_home ?x - animal)
(at ?x - animal ?l - location)
```

All the above are binary predicates, they are either true or false. They can have or not have variables. If `p` is a predicate, `(not (p))` refers to its negation.

Actions

```
(:action catch
  :parameters (?c - cat ?m - mouse)
  :precondition (and (at_home ?m) (at_home ?c))
  :effect (and
    (not (at_home ?m))
  )
)
```

An action normally contains a name, 0 or several parameters, 0 or several preconditions and several effects.

The example here is an action named “catch”. `?c` and `?m` are parameters for this action. The precondition says a cat `?c` must `at_home` and a mouse `?m` must `at_home`. When an object of cat and an object of mouse satisfy preconditions, the effect is that mouse `?m` no longer `at_home`.

Disjunctive conditions

The conditions can be generalised to any logical expression. Supported expressions include:

- `(not Condition)`
- `(and Condition_1 ... Condition_N)`